

Universidade Federal Rural do Semi-Árido
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I
Semestre: 2026.2
Data: ____/____/_____
Professora: Rosana Rego

— Exercício 9: Unidade I —

TAD: Tipo Abstrato de Dados (Inventario opaco)

Nome:_____ **Nota:**_____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade9.c`, atividade final da unidade, que evolui `atividade8.c` demonstrando, de forma visual e interativa (usando a biblioteca **raylib**), o conceito mais importante desta unidade:

- **TAD (Tipo Abstrato de Dados)**: um tipo cuja *struct* interna é **escondida** (tipo opaco), só manipulável através de funções públicas;
- **Encapsulamento**/ocultação de informação: quem usa o TAD não sabe (nem precisa saber) como ele é implementado por dentro;
- A diferença entre um **módulo comum** (`entidade.h`, *struct* visível, atividade 8) e um **TAD de verdade** (`inventario.h`, *struct* escondida);
- Revisão geral: ponteiros, alocação dinâmica, *struct*, **enum**, **union** e vetor de ponteiros para *struct* – agora esses últimos ficam completamente escondidos dentro do TAD.

O jogo é o mesmo de sempre, mas a coleção de entidades deixou de ser um vetor de ponteiros manipulado diretamente pelo `main` e passou a ser um TAD chamado `Inventario`.

1 O cabeçalho opaco inventario.h

Ao contrário de `entidade.h`, o cabeçalho `inventario.h` **não revela nenhum campo** da *struct*: ele apenas declara um **tipo incompleto** e as funções da interface pública:

```c
typedef struct inventario Inventario; // tipo opaco: struct definida so em inventario.c

Inventario *inventarioCriar(int capacidade);
void        inventarioDestruir(Inventario *inv);

bool        inventarioAdicionar(Inventario *inv, Entidade *e);
void        inventarioRemover(Inventario *inv, int indice);

Entidade   *inventarioObter(Inventario *inv, int indice);
int         inventarioTotal(Inventario *inv);
```

Quem inclui esse cabeçalho só enxerga a existência de um `Inventario *`: não é possível escrever `inv->total` ou `inv->vetor` fora de `inventario.c` – o compilador nem sequer conhece o tamanho dessa *struct* em outro arquivo.

2 A struct escondida em inventario.c

Somente dentro de `inventario.c` a *struct* é definida por completo. A representação escolhida é um vetor de ponteiros para `Entidade` (igual à atividade 5), mas isso é um **detalhe de implementação**:

```c
struct inventario {
    Entidade **vetor;       // vetor de ponteiros para Entidade, alocado dinamicamente
    int         total;
    int         capacidade;
};
```

Se um dia essa representação for trocada por uma lista encadeada, por exemplo, **nenhuma linha** de `atividade9.c` precisaria mudar – só o conteúdo de `inventario.c`.

3 Funções da interface pública

Todas as operações sobre o inventário passam por funções que recebem e devolvem apenas o ponteiro opaco ou tipos já conhecidos (`Entidade *`, `int`, `bool`):

```c
bool inventarioAdicionar(Inventario *inv, Entidade *e) {
    if (inv == NULL || e == NULL || inv->total >= inv->capacidade)
        return false;

    inv->vetor[inv->total] = e;
    inv->total++;
    return true;
}

void inventarioRemover(Inventario *inv, int indice) {
    if (inv == NULL || indice < 0 || indice >= inv->total) return;

    free(inv->vetor[indice]);
    inv->vetor[indice] = inv->vetor[inv->total - 1];
    inv->total--;
}
```

4 atividade9.c: usando apenas a interface pública

O programa principal nunca acessa campos internos do `Inventario` – só chama as funções declaradas em `inventario.h`:

```c
Inventario *inventario = inventarioCriar(MAX_ENTIDADES);
inventarioAdicionar(inventario, jogador);

for (int i = 1; i < inventarioTotal(inventario); i++) {
    Entidade *e = inventarioObter(inventario, i);
    if (entidadeColidiu(jogador, e)) {
        /* ... */
    }
}

inventarioDestruir(inventario); // uma unica chamada libera tudo
```

Módulo comum *vs.* TAD

Aspecto	**Módulo (`entidade.h`)**	**TAD (`inventario.h`)**
Struct visível para quem usa	sim (`e->pos`, `e->vida`)	não (tipo opaco)
Acesso aos dados	direto, pelos campos	só por funções (`inventarioObter`, ...)
Trocar a implementação depois	pode quebrar quem usa	não quebra quem usa

Essa é a ideia central de um TAD: **esconder** a representação interna para que ela possa evoluir livremente, desde que a interface pública continue a mesma.

5 Exercícios

Exercício 1 — Verificando se o inventário está cheio

Atualmente `inventarioAdicionar` apenas retorna `false` silenciosamente quando não há espaço.
Tarefa: adicione `bool inventarioEstaCheio(Inventario *inv)` à interface pública (declare em `inventario.h`, implemente em `inventario.c`) e use-a em `atividade9.c` para exibir um aviso na tela quando o inventário estiver cheio.
Dicas:

- A função deve comparar `inv->total` com `inv->capacidade` – só é possível escrever essa comparação **dentro** de `inventario.c`.
- Em `atividade9.c`, chame apenas `inventarioEstaCheio(inventario)`, sem acessar nenhum campo diretamente.

Exercício 2 — Contando entidades por tipo

Atualmente não há como saber, de fora do TAD, quantas entidades de um determinado tipo existem sem percorrer manualmente todo o inventário.
Tarefa: implemente `int inventarioContarPorTipo(Inventario *inv, TipoEntidade tipo)`, que devolve quantas entidades do tipo informado estão armazenadas.
Dicas:

- Implemente a função dentro de `inventario.c`, percorrendo `inv->vetor` internamente (isso só é possível ali, pois é o único arquivo que conhece a struct).
- Declare o protótipo em `inventario.h` para que `atividade9.c` possa chamá-la.
- Use essa função para exibir, na tela, quantos itens (`ENTIDADE_ITEM`) ainda restam no mapa.